

Task-Spec

The complete handbook for an atomic, vendor-portable, self-verifying unit of agentic work.

Intent becomes behavior. Behavior becomes executable proof. Proof becomes a bounded handoff.

This handbook follows one Task-Spec from authoring through pre-dispatch certification, execution, post-execution acceptance, backlog recovery, and Converge Passes 6–8.

READ THIS FIRST

Two versions, two jobs

The package release and the file-format contract are deliberately independent.

Package release: 0.1.0

The repository, `cvg` CLI, eleven bundled skills, and this handbook ship together as Converge **0.1.0**. The root `VERSION`, `cvg version`, and current changelog are the release authorities.

Format contract: 3

Every current Task-Spec declares `format_version: 3`. That value tells parsers which fields, zones, traceability rules, and compatibility behavior apply. It does not mirror the package version.

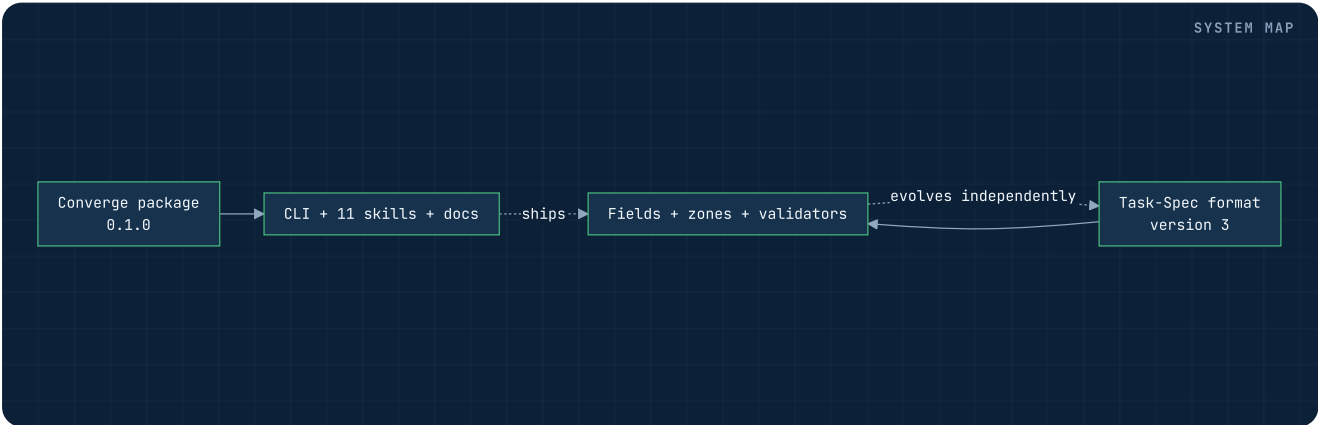
A future Converge 0.2 release could still read format 3. Conversely, a future additive format revision would not require pretending the entire package has the same number. Keeping these axes separate makes migrations explicit and lets old specs remain readable.

Evidence order used in this handbook. Executable scripts and current `cvg` behavior outrank skill instructions; current skill references outrank planning prose; legacy PDFs and schemas are visual or historical evidence only. Where old prose says “minimal,” “GLM only,” “four zones,” or `hmac-sha256-v1`, this handbook records the implemented behavior: `lite`, a configurable long-horizon backend set, six profile-scaled zones, and the v2 authorization envelope.

Scope boundary: this is the end-to-end Task-Spec contract. It does not claim that every proposed fleet, graph, or manager command is implemented.

IDENTIFIER	OWNER	CHANGES WHEN
<code>0.1.0</code>	Root package release	CLI/skills/package release is cut
<code>format_version: 3</code>	Task-Spec parser contract	A file-shape compatibility boundary changes
<code>agent_contract.version: 2</code>	Executor interface	The typed runner contract changes
<code>hmac-sha256-v2</code>	Sign-off serializer	The sealed authorization payload changes

These identifiers are not aliases. A validator may accept an older file contract with warnings while the package is current; a v1 HMAC may verify on its own narrow payload while still being downgraded to supervised Tier 2. Compatibility preserves readability—it does not silently grant current autonomy.



MAP

Contents

A contract, its proofs, and the operating system around it.

01 · Contract anatomy

- 1. Definition and non-goals
- 2. Six-zone file anatomy
- 3. Frontmatter: identity, authority, provenance
- 4. Profiles: lite, standard, full
- 5. Intent, Behavior, Contract, Guardrails, Operations, Reversal

02 · Scale and dependency

- 6. XS-L runnable leaves
- 7. XL/XXL decomposition nodes
- 8. Dependency DAG and ready frontier
- 9. Lifecycle and A2A mapping

03 · Trust gates

- 10. Structural validation
- 11. PRE gate and autonomy stamp
- 12. HMAC v2 authorization boundary
- 13. Tier 1/2/3 trust policy
- 14. Executor conformance and POST acceptance

04 · Operations

- 15. Five-layer backlog
- 16. Register → Bind → Loop
- 17. Authoring workflow and command cookbook
- 18. Troubleshooting and implementation ledger
- 19. Worked example, glossary, source map

Pages are organized as a descent: first define the artifact, then prove it is safe to delegate, then prove the resulting work is actually done.

THE UNIT

One bounded promise

A Task-Spec is executable acceptance, not a prettier ticket.

A Task-Spec is one Markdown file that defines a bounded unit of work in a form a conformant human or software executor can pick up, execute, verify, and report. Its center is not the prose. Its center is a set of runnable evaluations and a final Exit Check that return success only when the promised behavior exists.

Five non-negotiables

- **Atomic:** one coherent done-condition per leaf.
- **Portable:** executor-neutral input and typed output.
- **Self-verifying:** proof travels with the task.
- **Pickupable:** no hidden context required to begin.
- **Reportable:** lifecycle and receipts are machine-readable.

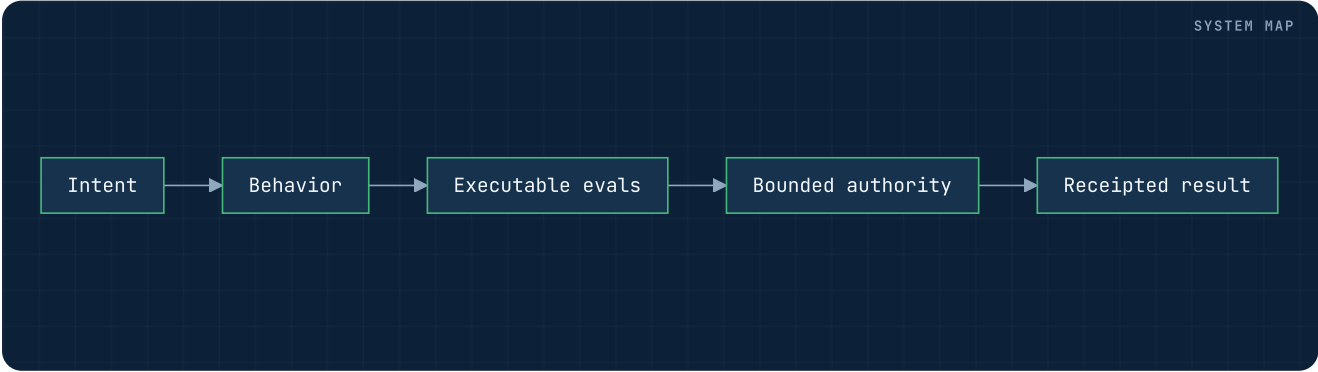
What it is not

- Not a PRD, ADR, or implementation plan.
- Not a prompt that grants unlimited repository authority.
- Not a declaration that work is complete.
- Not tied to one model vendor or harness.
- Not a substitute for a runtime sandbox or external reviewer.

The distinction between *specified*, *safe to delegate*, and *accepted* is load-bearing. A valid file may still be unsigned. A signed file authorizes execution but does not prove the implementation. In the shipped Converge path, Loop settles and attempts the matching pass receipt. Plain POST does not write Task-Spec or Converge metadata; only POST `--stamp` writes `accepted: true`. Lifecycle may advance to `done` only through the explicit locked transition after both forms of evidence exist.

Short form: Task-Spec turns “please do this” into “make these observable behaviors true, inside this authority boundary, and stop under these conditions.”

ARTIFACT STATE	WHAT IS KNOWN	WHAT IS NOT YET KNOWN
Authored	Intent and proposed proof are written	Whether the contract is structurally sound
Valid	The current linter accepts its shape	Whether it is dispatch-authorized
Signed	PRE certified eval mechanics and recorded a trust tier	Whether implementation is complete
Settled	Loop or a harness reported settled work	Whether the matching pass receipt exists and POST accepts the result
Accepted	POST reran proof, checked scope when enabled and Git data was available, and stamped the envelope	Whether scope was skipped and whether the locked lifecycle transition has run
Done	Acceptance is stamped, <code>cvg/receipts/<id>.json</code> matches the task with <code>result: pass</code> , and the lifecycle transition succeeded	Deployment health outside declared hooks



FILE ANATOMY

Six profile-scaled zones

Machine metadata surrounds a human-readable contract with executable proof at its center.

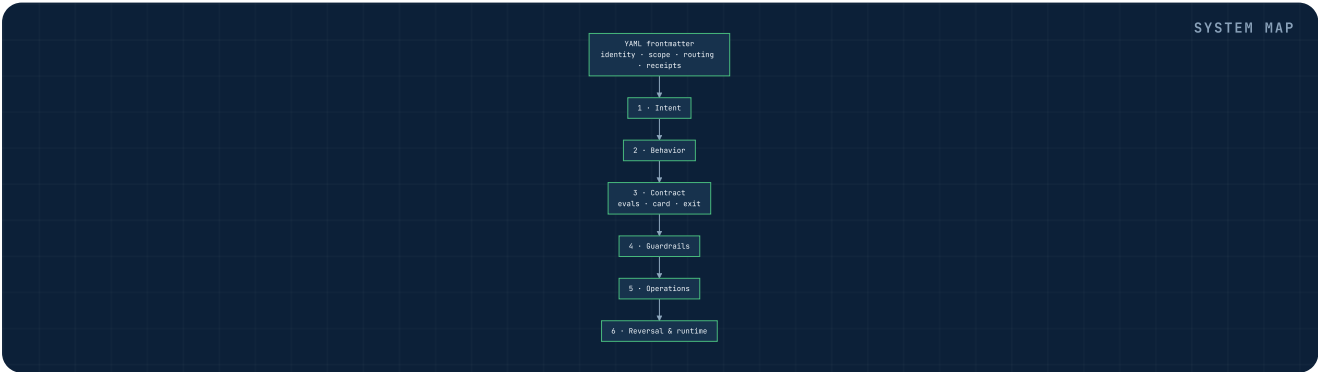
The canonical format is YAML frontmatter followed by six conceptual zones. “Six” describes responsibilities, not necessarily six headings: Rollback and Observability form one reversal-and-runtime zone, while the Contract zone contains three headings.

ZONE	QUESTION ANSWERED	CANONICAL CONTENT
1 • Intent	Why does this exist?	Why callout, Goal, Context
2 • Behavior	What must be observable?	Stable B-N Given/When/Then clauses
3 • Contract	How is done proven?	Success Criteria, Validation Card, Exit Check
4 • Guardrails	What must not happen?	Anti-Patterns, Do-Not-Touch
5 • Operations	What is unknown or blocking?	Open Questions, honest unknowns, and blocker conditions
6 • Reversal & runtime	How do we reverse and observe?	Rollback Plan, Observability Hooks

The irreducible core is **Goal + runnable Success Criteria + Validation Card + Exit Check**. A `lite` task may omit the surrounding explanatory zones. `standard` adds Behavior, Context, Guardrails, and Operations. `full` also requires Rollback and Observability.

Drift repaired: some retained prose still uses “four zones” while listing six, or numbers Behavior and Contract inconsistently. The validator’s profile rules and current format reference establish the grouping shown here.

ZONE	PRIMARY AUTHOR	PRIMARY CONSUMER	MACHINE CHECK
Intent	Task architect	Worker/reviewer	Required headings by profile
Behavior	Task architect + domain owner	Reviewer and eval designer	B-N presence and references
Contract	Task architect	Gate, worker, verifier	Functions, card, Exit Check
Guardrails	Repo/domain owner	Worker and POST diff gate	Section presence; scope checked later
Operations	Author/reviewer	Dispatcher	Open-question section; blocked state elsewhere
Reversal/runtime	Operator	Harness/on-call	Required for full; enforcement is external



FRONTMATTER I

Identity and lifecycle fields

The file can be found, claimed, bounded, and recovered without parsing prose.

FIELD	ROLE	RULE IN CURRENT FORMAT
<code>id</code>	Stable identity	<code>T-YYYYMMDD-kebab-slug</code> ; matches filename
<code>title</code>	Human label	One concrete unit of work
<code>format_version</code>	Parser contract	Current authored files use integer <code>3</code>
<code>profile</code>	Ceremony level	<code>lite standard full</code> ; absent resolves to standard
<code>status</code>	Lifecycle truth	<code>ready in-progress blocked done parked</code>
<code>created</code>	Audit time	ISO-8601 timestamp
<code>tags</code>	Navigation	Optional taxonomy labels

For a current format-3 leaf, the validator expects the identity and execution core to include `id`, `title`, `status`, `effort`, `budget_iterations`, `agent`, `depends_on`, `touches_paths`, `source_note`, and `created`. The generator also writes `format_version` and `profile`; omitting the former invokes legacy compatibility behavior rather than creating a well-formed new v3 task.

Identity is intentionally deterministic. It lets dependencies, issue projections, metrics events, receipts, and archive paths refer to the same unit without relying on a mutable title. Status is per-file truth; derived indexes may be rebuilt from it.

Never use status as a success claim by itself. `done` is a lifecycle state. It is legal only after the POST acceptance envelope is stamped and `cvg/receipts/<id>.json` names the same task with `result: pass`.

Defaults and write ownership

- `profile` defaults to `standard` when absent; current generators still write it explicitly.
- Authors should leave `status: ready` only when no open hole remains; the generator cannot infer that semantic fact. The transition tool—not the worker’s editor—owns later lifecycle writes.
- `signed_off*` belongs exclusively to the PRE gate; `accepted*` belongs exclusively to the POST gate.
- `format_version`, `id`, and `created` are authoring-time identity. Changing them after registration creates a new audit problem and requires full revalidation.

The parser treats a missing format number as legacy input. That tolerance exists for migration; a new task should never depend on it.

FRONTMATTER II

Scope, routing, provenance, receipts

Fields are grouped by the decision they control—not by when they were added.

GROUP	FIELDS	MEANING
Write authority	<code>touches_paths</code> , <code>creates_paths</code> , Do-Not-Touch	Allowed changes and explicit exclusions
Graph	<code>parent</code> , <code>depends_on</code> , <code>children</code>	Origin, predecessor edges, node expansion
Routing	<code>effort</code> , <code>agent</code> , <code>execution_backend</code>	Size, capability hint, executor name
Budgets	<code>budget_iterations</code> , optional <code>budget_tokens</code>	Finite retry and resource authority
Provenance	<code>source_note</code> , <code>source_action_item</code>	Why the task exists and where it came from
Accountability	<code>owner</code> , <code>priority</code> , <code>severity</code> , <code>due_date</code>	Human responsibility, urgency, consequence, timing
Readiness	<code>precondition</code> , <code>blocked_reason</code> , <code>requires</code>	External preconditions, holes, sandbox requirements
Receipts	<code>tracker_ref</code> , sign-off and acceptance envelopes	Projection backlink and gate outcomes

`execution_backend` is an open, single-token string. Built-in names are conveniences, not a closed vendor enum. Executor-specific options belong under the Validation Card's `backend_metadata` , keeping the top-level contract portable.

`tracker_ref` is a convenience backlink written after an optional Register pass. The issue-side marker is the idempotency key, so a missing backlink does not invalidate delegation. Mutable receipts such as `tracker_ref` , `status` , and `accepted` are deliberately excluded from the PRE HMAC; fields that grant execution authority are sealed.

Who may write what

OWNER	FIELDS	DEFAULT / SENTINEL
Author	<code>parent</code> , <code>scope</code> , <code>graph</code> , <code>effort</code> , <code>agent</code> , <code>backend</code> , <code>budgets</code> , <code>provenance</code>	Lists use <code>[]</code> ; optional scalars use <code>(none)</code>
Register	<code>tracker_ref</code>	<code>(none)</code> until projection succeeds
Lifecycle tool	<code>status</code>	Transition reason is appended to <code>_metrics.jsonl</code> ; ordinary transitions do not populate <code>blocked_reason</code>
PRE gate	<code>signed_off</code> , <code>signed_off_by</code> , <code>signed_off_at</code> , <code>signed_off_sig</code>	false / none / no sig
POST gate	<code>accepted</code> , <code>accepted_by</code> , <code>accepted_at</code>	false / none

`priority` is urgency (P0–P4); `severity` is consequence. A low-urgency financial task may still require a full profile. `precondition` is an external fact, while `depends_on` points to another Task-Spec.

PROFILES

Rigor scales with consequence

Profiles reduce ceremony without weakening executable proof.

PROFILE	REQUIRED CONTENT	USE WHEN
lite	Goal, Success Criteria, Validation Card, Exit Check	Small, low-blast-radius work where evals express the behavior directly
standard	lite + Context, Behavior, Anti-Patterns, Do-Not-Touch, Open Questions	Default; work another person or executor must understand
full	standard + Rollback Plan + Observability Hooks	Unattended, hard-to-reverse, security, or financial-critical work

The profile axis is orthogonal to `format_version`. A v3 file without an explicit profile resolves to `standard` for backward compatibility, but current authoring should state the choice. The old name `minimal` is stale; the implemented value is `lite`.

Profiles do not remove the moat. Every profile still carries runnable success criteria and a final Exit Check. What scales is the amount of explanatory context, behavioral traceability, reversal planning, and runtime observation around that proof.

Escalate to full

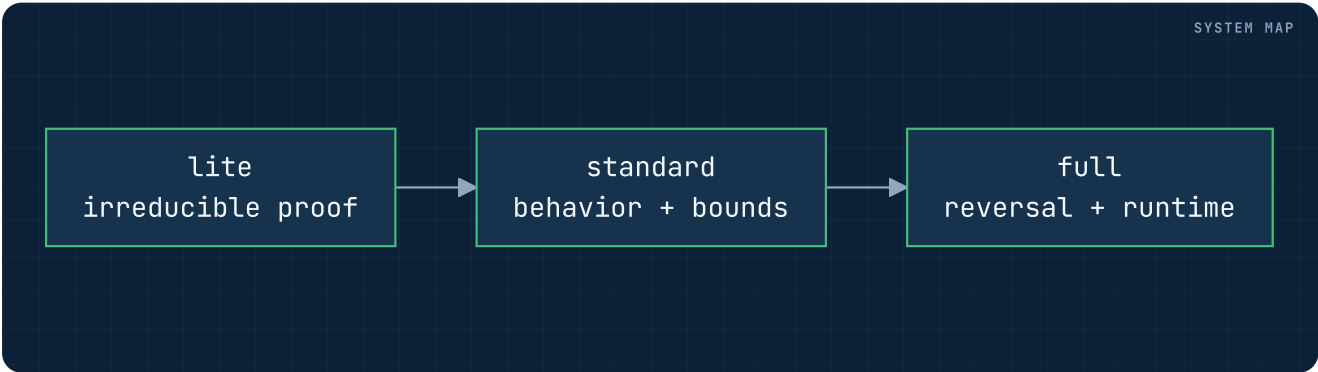
- Unattended or overnight execution
- Security or financial-critical severity
- Destructive or difficult rollback
- Runtime behavior needs explicit alerts

Do not over-escalate

- A one-line docs fix should not carry an ops runbook
- Atomicity is set by effort, not profile
- A full parent does not force every child to full
- Choose the lightest profile that still tells the truth

REQUIREMENT	LITE	STANDARD	FULL
Goal + executable Contract	Required	Required	Required
Context + Behavior	Optional	Required	Required
Behavior ⇔ eval traceability	Not enforced	Validator/PRE cover orphan behavior + dangling IDs; DoD adds missing eval mapping	Validator/PRE cover orphan behavior + dangling IDs; DoD adds missing eval mapping
Anti-Patterns + Do-Not-Touch + Open Questions	Optional	Required	Required
Rollback Plan + Observability Hooks	Recommended when useful	Recommended when useful	Required

The validator permits explicit “none” bodies where the requirement was considered and is genuinely inapplicable; it rejects a missing required heading. A full profile with no `requires.network` currently produces an operational warning, not a false claim of network enforcement.



ZONE 1 · INTENT

Enough why—without a context dump

Intent frames the atom; it does not recreate the feature specification.

Intent has three parts: a one-paragraph **Why** callout, a concrete **Goal**, and—on standard/full—a lean **Context** section. Together they let an executor understand why this atom matters and where to look, without importing the parent PRD into the working context.

A strong Goal

Names one observable outcome, its affected surface, and the boundary of the work. “Add a health endpoint that returns HTTP 200 and a fixed JSON body” is executable.

A weak Goal

Names activity or aspiration. “Improve reliability,” “work on the API,” and “make health checks better” cannot drive an Exit Check.

The `parent` field preserves feature-level lineage. It points to the PRD, ADR set, plan, or external URL the atom distills; the detail is referenced, not copied. `source_note` records the originating evidence such as a meeting note, audit, or issue. These are different: `parent` is altitude lineage, `source` is provenance.

Context should name only the files, interfaces, constraints, and prior decisions needed to begin. Link to canonical documentation. If Intent is longer than Behavior and Contract combined, the unit is probably too broad or the author is using the task file as a PRD.

Open information is not context. If a missing fact would change the implementation or proof, surface it as an Open Question and set the task to `blocked`; do not invite the executor to guess.

INTENT ELEMENT	GOOD TEST	FAILURE SIGNAL
Why	Explains the user/system consequence in one paragraph	Only repeats the title
Goal	Can be restated as one terminal outcome	Uses “improve,” “support,” or “work on” without a threshold
Context	Points to live interfaces and decisions	Pastes an entire PRD or stale investigation
Parent	Resolves to the feature-level authority	Embeds feature scope into the atom
Source note	Lets a reviewer recover provenance	Names only a chat memory or undocumented request

A reviewer should be able to delete Context and still understand success from Behavior and Contract. Context makes execution efficient; it must not be the only place a requirement is stated.

ZONE 2 · BEHAVIOR

Stable statements before shell

Given/When/Then clauses preserve meaning while implementations and test commands evolve.

Behavior is the human-readable layer between intent and executable evaluation. Each statement receives a stable identifier such as `B-1` and follows a Given/When/Then shape:

```
- B-1 - GIVEN the API is running
  WHEN a client requests /health
  THEN the response is HTTP 200 with {"status":"ok"}.
```

The clause names an observable outcome, not an implementation. “Add a FastAPI route decorator” is a design choice; the client-visible response is behavior. Stable behavior IDs let the Validation Card say exactly which promises each eval proves.

Good behavior

- Observable at a boundary
- Specific enough to falsify
- Independent of model or harness
- One promise per B-N

Behavior smells

- “Code is clean” or “UX looks good”
- Several unrelated outcomes joined by “and”
- A file edit presented as user behavior
- A requirement no eval can observe

Behavior is required for `standard` and `full` format-3 tasks. A `lite` spec may omit it; its behavior is implied directly by the evals. That exception is a deliberate compact mode, not permission to use vague proof.

Design rule: write Behavior before shell. If a B-N cannot be mapped to deterministic evidence, refine the behavior or declare a human/LLM checkpoint honestly rather than hiding subjectivity inside a command.

```
# Strong: stable outcome
B-2 - GIVEN an unknown account id
  WHEN the lookup endpoint is called
  THEN it returns 404 with error code ACCOUNT_NOT_FOUND.

# Weak: implementation instruction
B-2 - Add an if statement in handlers/accounts.py.
```

The strong clause permits refactoring while preserving the contract. Its focused eval can call the public interface; a separate regression eval can cover the existing success path. If a behavior has several independently failing outcomes, split it into stable IDs so verification localizes the defect. IDs should survive wording improvements; renumber only when the contract itself changes and then revalidate every `verifies` reference.

ZONE 3 · CONTRACT

Success Criteria are executable evidence

Each eval is a terminal, repeatable question with a process exit code for an answer.

The Success Criteria section defines shell functions named `eval_1()`, `eval_2()`, and so on. Exit code zero means the criterion is true; non-zero means it is not. The validator requires at least one `eval_N` function. The standard template uses three because layered proof—startup, focused behavior, regression—is usually stronger than a single broad test.

Order checks from cheap and local to expensive and integrated. A useful stack is:

1. **Structural:** the file, symbol, schema, or command exists.
2. **Behavioral:** the exact promised outcome is observed.
3. **Regression:** the relevant suite still passes.

Properties of a strong eval

- Deterministic or explicitly classified otherwise
- Idempotent and safe to rerun
- Terminal: it decides, not merely reports
- Scoped to the promised behavior
- Returns non-zero on a real failure

Common false proof

- Checking only that a string exists
- Suppressing failure with unconditional `|| true`
- Reading mutable external state without pinning it
- Asserting implementation rather than behavior
- A command that passes before the work is applied

`check_type: deterministic` is preferred. `llm_judge` is available for genuinely subjective criteria, but the judgment prompt and expected evidence must be explicit. “Looks good” is not a contract.

```
# Focused behavior: non-zero unless the exact JSON is returned
eval_2() {
  body="$(curl -fsS http://127.0.0.1:8001/health)" || return 1
  printf '%s' "$body" | jq -e '.status = "ok"'
}

# Regression: execute the artifact rather than grep for a stub
eval_3() {
  pytest -q tests/test_health.py
}
```

Quote variables, use bounded timeouts for processes and networks, and clean up background services with traps. Prefer a tool’s native exit status over capturing human-formatted output. An existence check may be valid for a document deliverable, but for code it is only evidence that a stub exists; the PRE gate escalates an all-existence-only proof set unless explicitly and honestly allowed.

VALIDATION CARD

The typed execution interface

The card mirrors proof, retry policy, and executor obligations in machine-readable YAML.

The Validation Card lives inside the Markdown file but is structured YAML. It contains three cooperating contracts:

BLOCK	REQUIRED DECISIONS
success_criteria	Eval ID, description, runner, check type, B-N coverage, terminal flag, expected duration
retry_policy	Maximum iterations, no-progress breaker, terminal-failure action
agent_contract	Contract version, zones to read, artifacts to produce, tools, timeout, sandbox, emits

```
agent_contract:
  version: 2
  read: [intent, behavior, contract, guardrails, operations]
  produce: [code, tests]
  required_tools: [git, bash]
  timeout_minutes: 30
  sandbox_type: isolated
  output_artifacts: []
  mcp_dependencies: []
  emit: [pass, fail, retry_with_reason, parked_with_context]
  backend_metadata: {}
```

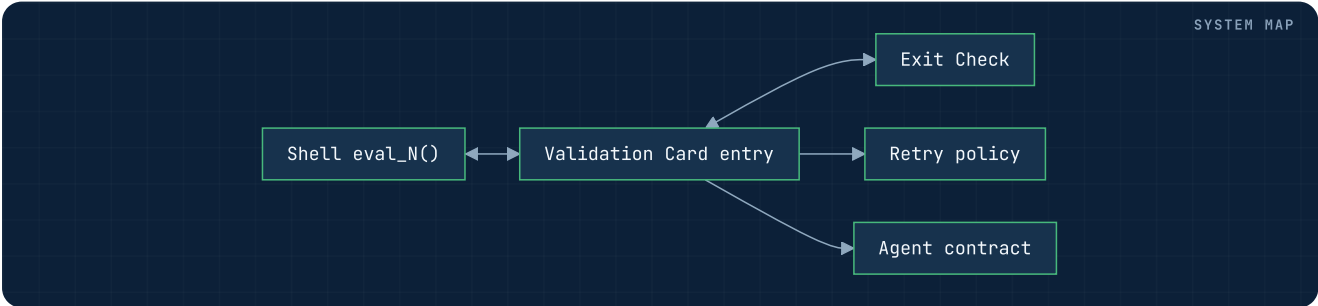
`agent_contract.version: 2` is another independent contract identifier; it is not the package or Task-Spec format number. `produce` and `emit` are YAML lists in the current contract. `backend_metadata` is an open nested object where an executor may namespace its own options without turning the whole format into a vendor enum.

The card must agree with the shell. An eval listed in YAML but absent from Success Criteria is fiction; a shell eval absent from the card loses traceability and duration metadata. The Exit Check closes the three-way correspondence.

CARD FIELD	OWNERSHIP / CONSTRAINT
max_iterations	Mirrors the frontmatter retry budget; a dispatcher must not raise it silently
circuit_breaker_no_progress	Stops repeated non-improving attempts before the outer maximum
timeout_minutes	Positive integer, maximum 1440 in the typed contract
sandbox_type	host isolated ephemeral ; a requested value is not proof the runtime supplied it
required_tools	Non-empty list the provisioner must satisfy or block pickup
emit	Non-empty subset of the allowed result vocabulary

`output_artifacts` and `mcp_dependencies` are optional. If present, they make capture and tool provisioning explicit; they do not expand write authority beyond frontmatter scope.

0.1.0 parser gap. Canonical timeout and no-progress values live in the HMAC-covered body. Loop’s first-match numeric parser can nevertheless consume unsigned same-name frontmatter overrides; those names are not in `TS_AUTHZ_FIELDS`. Do not duplicate them in frontmatter, and re-bind after any byte change.



EXIT CHECK

One final proof-of-done

The Exit Check composes every required criterion into one unambiguous terminal signal.

The Exit Check is a bash expression that returns zero only when the entire task contract is satisfied. For independent required evals, the canonical shape is simple:

```
eval_1 && eval_2 && eval_3
```

Every defined criterion that participates in “done” must appear. The gate checks the functions and the Exit Check together so an author cannot define a convincing eval and accidentally omit it from the terminal decision.

PRE interpretation

Before implementation, assertion failure is normal—the promised work may not exist. The PRE gate is looking for broken evaluation mechanics: syntax errors, missing commands, unbound variables, malformed numeric tests, or an Exit Check that cannot run.

POST interpretation

After execution, every required eval must return zero. At this point a failure means the work is incomplete or the environment cannot prove it. The POST gate rejects the completion claim.

Keep the check idempotent. It may run during authoring, every loop iteration, independent verification, acceptance, recovery, and CI. Avoid commands that mutate production state, rely on ambient credentials without declaring them, or leave background processes behind.

Goodhart guard: with `accept-task.sh --gold-sanity`, the current eval set must fail on the unpatched baseline and pass on the current work. An always-green Exit Check is not evidence.

Safe composition

```
eval_1 &&  
eval_2 &&  
eval_3
```

Short-circuits at the first failure and preserves the failing status.

False green

```
eval_1 || true  
eval_2  
echo "done"
```

The final echo returns zero and erases prior failure.

The runner extracts functions from the designated section; avoid relying on interactive shell startup, unstated working directories, or state created by an earlier manual command. If sequence matters, make setup explicit and ensure cleanup happens on both success and failure.

TRACEABILITY

Every promise must have proof

Validator/PRE and DoD cover different halves; complete traceability requires both.

Traceability is executable policy for format-3 `standard` and `full` specs, but enforcement is deliberately split. Structural validation and PRE hard-fail a declared `B-N` with no proving eval and any `verifies` reference to an undeclared behavior. An eval without `verifies` is a warning there. `cvg tasks dod` hard-fails orphan behaviors and evals missing `verifies`, but does not reject dangling IDs. Full bidirectional coverage requires both. In release 0.1.0, both trace extractors recognize only inline bracket syntax such as `verifies: [B-1, B-2]`; block-style `verifies` lists are not parsed.

DEFECT	WHY IT MATTERS	REPAIR
Orphan behavior	A promise exists with no executable evidence	Add a proving eval or remove/refine the promise
Dangling reference	An eval claims to prove a nonexistent requirement	Correct the ID or declare the behavior
Unmapped eval	Work is being checked without a stated reason	Add <code>verifies</code> or use lite deliberately
Overloaded eval	Failure does not localize which promise broke	Split the check along behavior boundaries

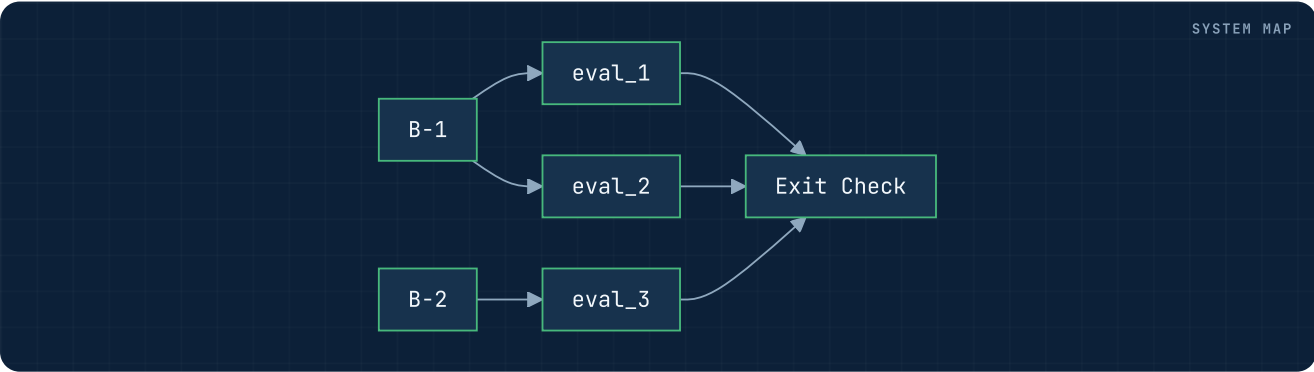
Many-to-many mappings are valid. One integration eval may prove several behaviors, and one behavior may deserve both a focused check and a regression test. The goal is complete, legible coverage—not forcing a one-to-one shape.

Traceability gives review a concrete question: “Show me the eval for B-2.” It also makes change safer. If behavior is amended after sign-off, the body digest changes and a keyed HMAC fails. The spec must be reviewed and re-stamped rather than silently widening the definition of done.

Review trace in both directions

1. Enumerate every B-N from the Behavior section.
2. For each one, locate at least one Validation Card `verifies` entry and the corresponding shell function.
3. Enumerate every deterministic `eval_N` and explain which user/system promise requires it.
4. Confirm the Exit Check actually includes every required function.

Use structural validation while authoring, PRE at the delegation boundary, and `cvg tasks dod` when the release decision requires complete Behavior $\rightleftharpoons$ eval coverage. Neither check alone establishes both directions.



ZONES 4-5

Guardrails and honest unknowns

The contract says not only what to achieve, but what authority is withheld.

Guardrails

Anti-Patterns name specific tempting actions and why they are wrong. **Do-Not-Touch** names paths or surfaces the executor must not modify. “Be careful” is not a guardrail; “do not edit the migration history—add a new migration” is.

Operations

Open Questions records unresolved facts. If an answer changes implementation, scope, or proof, the task is `blocked` with a `blocked_reason`. Unknowns do not become permission for autonomous guessing.

Write authority has two positive lists: `touches_paths` for existing surfaces and `creates_paths` for greenfield files. The POST blast-radius check compares the Git-visible repository change set against their union, then separately rejects anything matching Do-Not-Touch. Ignored bytes and out-of-repository effects require installed runtime or OS controls. Declaring a file under both allowed and forbidden surfaces does not make it safe; the explicit prohibition wins.

Guardrails should be operationally testable where possible. A path exclusion can be diffed. A forbidden API dependency can be searched. A requirement to preserve public behavior can be represented as a regression eval. Prose is still useful for judgment, but executable constraints are harder to overlook.

Required emptiness is explicit. Standard/full sections that genuinely have nothing to add should say `(none)` with a reason. Omission and considered absence are different states.

POST scope algorithm

1. Resolve the Git change set against the chosen base reference.
2. Build the allowed set from block-style `touches_paths` / `creates_paths` entries. Release 0.1.0 POST does not parse inline frontmatter lists here even though validation does; inline lists can leave the allowed set empty and cause false blast-radius rejection.
3. Reject any changed path outside that union.
4. Reject any changed path matching Do-Not-Touch, even if also allowed.
5. Require re-authoring and a fresh PRE stamp before intentionally widening scope.

`--no-blast-radius` exists for an explicitly broad or non-Git context, but it removes a central independent check and is reported as skipped. It must not become the default response to a poorly bounded contract.

ZONE 6

Reversal, observability, isolation

Full-profile work carries an operating envelope, not merely a test suite.

A `full` profile requires a Rollback Plan and Observability Hooks. Rollback names the concrete path back to the pre-task state: commits to revert, files or data to restore, feature flags to disable, and task state to record. Observability names expected duration, key metrics, log tails, and alert conditions that reveal a stalled or harmful execution.

The optional frontmatter `requires` block documents sandbox assumptions:

```
requires:
  base_image: python:3.13-slim
  deps: lockfile
  network: deny
```

The bash acceptance tool reports these requirements but cannot itself enforce container identity or network egress. Enforcement belongs to the dispatcher or sandbox. For a full task with missing network posture, the current implementation warns rather than claiming isolation.

SURFACE	TASK-SPEC RECORDS	RUNTIME MUST ENFORCE
Recovery	Rollback steps and state transition	Privileges, backups, deployment controls
Runtime signal	Metric, duration, alert threshold	Logs, telemetry, alert delivery
Isolation	Base image, dependencies, egress posture	Sandbox image, mounts, network policy

No false enforcement claim: a declared `network: deny` is a requirement on the runtime. The file and acceptance report make it auditable; only the harness can make it real.

FULL-PROFILE QUESTION	CONCRETE ANSWER
How is code reversed?	Named commit/flag/migration reversal with ordering
How is data restored?	Backup identifier, restore command, and owner
How long should execution take?	Expected duration plus stall threshold
What reveals harm?	Metric/log, normal range, alert threshold, observation window
What external access is required?	Base image, pinned dependencies, and explicit egress posture

An additive task may state that rollback is unnecessary, but the statement should explain why no destructive state exists. “None” without reasoning is weaker than a two-line recovery decision.

EFFORT ENGINE

Six tiers, one path

Size controls whether a Task-Spec runs or decomposes; it never routes work outside the method.

Effort is based on context, verification shape, and write surface—not on a human time estimate. There are two kinds of Task-Spec:

TIERS	KIND	MEANING
XS, S, M, L	Runnable leaves	One coherent objective that fits a bounded execution and verification context
XL, XXL	Decomposition nodes	Composition units that name child Task-Specs; never dispatched directly

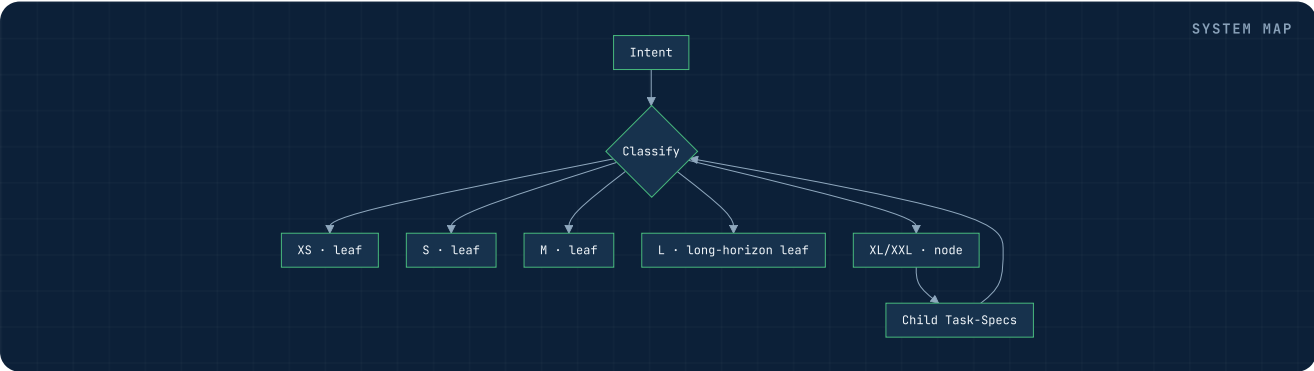
The method is “tasks all the way down.” Large work is not escaped into a different unverified format. An XL/XXL node expands into smaller Task-Specs until every executable leaf is XS-L and proves one coherent outcome.

The validator uses the union of touches_paths and creates_paths as a machine-visible proxy for blast radius. Read counts, eval counts, architectural layers, and context size remain authoring judgment. A budget breach is feedback that the atom is coarse, not an invitation to hide paths.

Effort is not profile. Effort answers “does this execute as one atom?” Profile answers “how much surrounding safety context must it carry?” An S financial-critical task can be full; an M docs task can be standard.

SIGNAL	CLASSIFICATION PRESSURE
One path and one focused predicate	XS/S leaf
Two or three tightly related paths and one boundary	S/M leaf
Several paths, one long-horizon done-condition	L ceiling
Multiple independently shippable outcomes or layers	XL node
Platform/backbone or several composition levels	XXL node

Classify upward when a surface is fragile, highly owned, or hard to reverse even if its path count is small. The machine budget is a floor for honesty, not a complete risk model.



RUNNABLE LEAVES

Write-surface budgets are a sizing signal

The validator counts this proxy and warns on excess; it does not hard-block delegation today.

TIER	WRITE PATHS	AUTHORING GUIDE	TYPICAL SHAPE
XS	≤ 1	One focused eval	Constant, config key, dependency bump
S	≤ 2	One or two evals	Small endpoint, one model plus test
M	≤ 3	Two or three evals	Related files, one bounded feature slice
L	≤ 5	Up to four evals, one coherent done-condition	One end-to-end service change on a long-horizon backend

The declared write surface is mechanically countable, and the validator compares it with the tier table. An excess currently emits a warning that PRE may still accept; treat it as an explicit split-or-reclassify review signal, not a hard guard. Read and eval counts are also guidance. By contrast, an L leaf’s configured long-horizon backend rule is enforced as an error.

Use the union of existing and new paths. A file created by the task goes under `creates_paths`; declaring it only under `touches_paths` causes path validation to ask whether the missing file should be treated as greenfield.

A leaf must carry one coherent failure story. If one eval can fail while the remaining work is independently shippable, the task probably contains multiple atoms. Split along architectural seams, independent test suites, or separately reversible outcomes.

L is the leaf ceiling. It still owns one done-condition and is accepted only when `execution_backend` belongs to the configured long-horizon set. The default set is `glm claude codex kimi` and can be replaced with `TASKSPEC_LONG_HORIZON_BACKENDS`. This is a capability policy, not the stale “GLM only” rule retained in some template/schema prose.

Do not optimize for a smaller letter. The tier is a safety classification. Hiding a path or merging unrelated evals makes the contract less truthful without making the work smaller.

When the validator reports a leaf over budget, three repairs are honest: split it, reclassify it upward within the leaf ceiling, or—when outcomes are independently verifiable—create an XL/XXL node. Expanding a glob until it conceals the real surface or moving changes to Do-Not-Touch are not repairs. For L, changing the backend name without confirming that runtime’s actual capability is equally cosmetic.

DECOMPOSITION NODES

XL and XXL do not execute

Large intent becomes a composition graph of independently gateable leaves.

XL and XXL files are decomposition directives. They own no write surface and are never handed to a worker as a runnable task. Their purpose is to name child Task-Specs and preserve the composition relationship.

TIER	MINIMUM CHILDREN	RUNNABLE?	WRITE SURFACE
XL	2	No	0; children own paths
XXL	3	No	0; children own paths

Split on natural seams: vertical feature slices, architectural layers, independent test suites, separately reversible changes, or ownership boundaries. Each child chooses its own effort and profile, declares its own authority, passes its own PRE gate, and earns its own acceptance envelope plus matching execution receipt.

The feature-level PRD or plan remains outside the atom and is referenced through `parent`. A human-skimmable flat index can list all children and edges. The execution graph comes from child `depends_on` fields, not from deeply nesting prose.

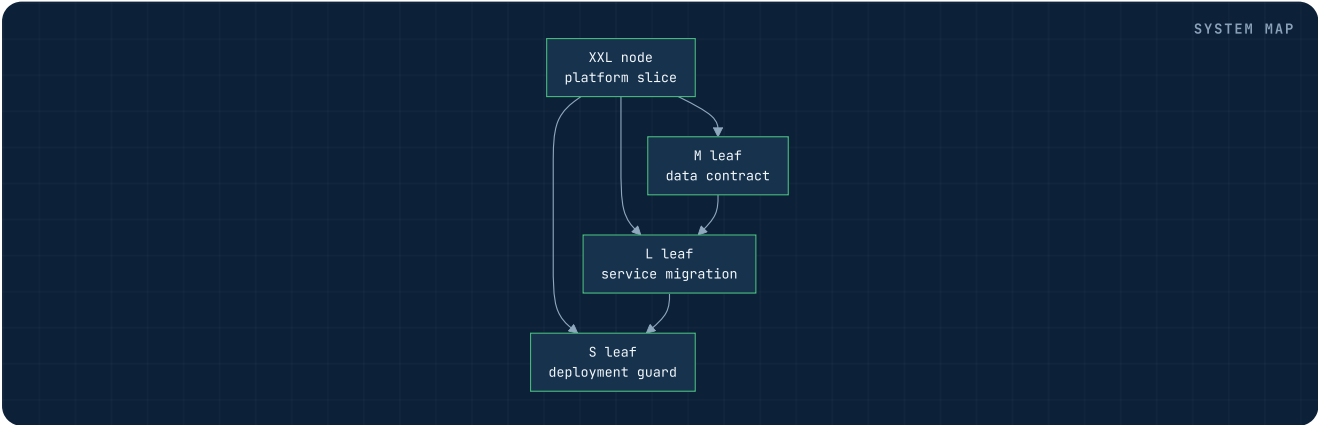
`cvg ready` includes a task when its status is `ready` and dependencies are done or legacy accepted. It does not validate leaf effort, PRE/trust, preconditions, or `blocked_reason`; dispatch policy must. Backlog linting separately resolves dependency IDs and detects cycles. A missing human fact is encoded separately as `status: blocked` plus `blocked_reason`; a precondition records an external condition. Neither should be hidden in prose or guessed by a worker.

No SDD escape hatch: the correction in the current method is “decompose into more Task-Specs,” not “route big work to a different format.” Composition happens after the children settle.

Node authoring invariant

The parent node carries `children` (at least two for XL, three for XXL), `touches_paths: []`, and no runnable claim. Each child carries a `parent` backlink, its own scope, behavior, proof, trust envelope, and lifecycle. Composition evidence should explain how child acceptance receipts satisfy the parent outcome; it must not replace their individual POST gates.

If children contend for the same file, sequence them with `depends_on` or redesign the slice. “Parallel” is never implied merely because tasks are siblings.



LIFECYCLE

State is explicit and recoverable

Five statuses describe work ownership; envelopes and receipts justify terminal lifecycle truth.

STATUS	MEANING	A2A V1.0 MAPPING
ready	Candidate state; <code>cvg ready</code> filters status + done/legacy-accepted dependencies only	<code>TASK_STATE_SUBMITTED</code>
in-progress	Claimed under a lock	<code>TASK_STATE_WORKING</code>
blocked	Needs input or environment repair	<code>TASK_STATE_INPUT_REQUIRED</code>
done	Stamped acceptance plus a matching passing execution receipt	<code>TASK_STATE_COMPLETED</code>
parked	Stopped with context, commonly budget exhaustion	<code>TASK_STATE_FAILED</code>

Executors should transition through the atomic status tool rather than editing YAML directly. Claiming moves `ready` → `in-progress` ; a missing external fact moves to `blocked` ; finite budget exhaustion moves to `parked` with a reason. The transition that actually changes status to `done` is different: it is refused until POST has stamped `accepted: true` and `cvg/receipts/<id>.json` carries the same `task_id` with `result: pass` .

Status and evidence are separate axes. `signed_off: true` records PRE certification; `accepted: true` records POST acceptance. Neither performs the lifecycle transition. The current `done` guard requires `accepted_by` and `accepted_at` values that are nonempty and not the literal `(none)` , plus a parseable receipt whose `task_id` exactly matches and whose `result` is `pass` . It does not validate receipt provenance, profile, hashes, or the full receipt schema; those remain conformance policy.

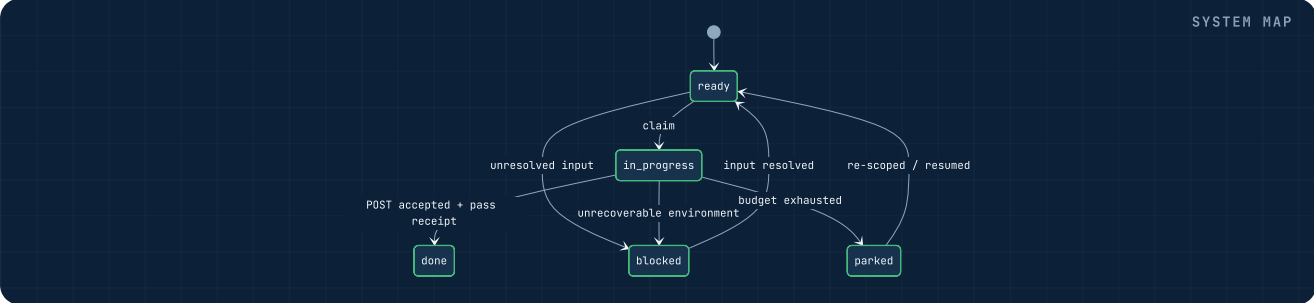
Recovery invariant: frontmatter status is authoritative per-file truth. If the derived index disagrees, rebuild the index from task files; do not rewrite tasks from the cache.

Recommended transition ownership

Release 0.1.0 serializes changes under a lock, validates the target is one of the five statuses, and treats same-state as an immediate no-op before the `done` evidence guard. For an actual state change whose target is `done` , it applies the acceptance/receipt guard. It does not re-audit an already-done file and does not reject other current-state → target-state combinations. The edge table below is operating policy, not an enforced matrix.

EDGE	WHO/WHY	REQUIRED CONTEXT
ready → in-progress	Executor claim under lock	Eligible deps, acceptable trust, satisfiable preconditions
in-progress → done	Lifecycle tool after independent POST and receipt creation	Accepted/by/at plus parseable matching <code>task_id</code> and <code>result: pass</code>
in-progress → parked	Budget stop	Reason and last useful context
ready/in-progress → blocked	Input/environment gap	Specific <code>blocked_reason</code>
blocked/parked → ready	Human/supervisor recovery	Gap resolved, scope and seal rechecked

The PRE seal does not cover `status` so legal progress does not invalidate authorization. It does cover dependencies and budgets, so changing the conditions of pickup or the amount of permitted work requires a new seal.



Validate before you certify

The linter establishes contract shape; it does not authorize unattended execution.

`validate-task-spec.sh` is the structural pre-gate linter. It checks the current format’s machine-visible invariants, including:

- Frontmatter delimiters, required fields, enums, and ID/filename match
- Profile-aware zone presence and unfilled placeholders
- Effort kind, write-surface budget, node child count, L backend policy
- Success functions, Validation Card, retry policy, and agent contract
- Behavior-to-eval traceability for standard/full v3
- Dependency resolution, cycles, parent paths, and greenfield paths
- Sign-off/acceptance envelope floors and HMAC verification when possible
- Known shell anti-patterns; optional shellcheck support

Exit zero means the file is structurally acceptable under the validator’s compatibility policy. Legacy formats may pass with warnings. Warnings must still be read: an L routing warning, Tier-2 trust notice, or undeclared isolation posture is operational information, not decorative output.

Validation does not run the entire PRE certification protocol and does not stamp the file. It refreshes derived `_state.yaml` by default; use `--no-state` for a read-only structural check. In particular, it cannot make a hand-edited `signed_off: true` trustworthy. The autonomy boundary belongs to `safe-to-delegate.sh --stamp`.

```
cvg tasks validate --no-state cvg/tasks/T-20260729-add-health-endpoint.md
# 0: valid / compatible-with-warnings
# non-zero: repair the listed contract defects
```

Separate questions: “Can I parse and reason about this contract?” is validation. “May a worker run it?” is the PRE gate. “Did the work become real?” is POST acceptance.

VALIDATOR RESULT	OPERATOR MEANING
Error / non-zero	Contract is not consumable; fix before any gate
Valid + warning	Shape is compatible, but policy or quality needs review
Valid, unsigned	Authoring may continue; no unattended dispatch authority
Valid, Tier-2 envelope	Supervised-only; provision key and re-stamp for Tier 1
Tier-3 mismatch	Hard stop; investigate body/authority mutation

Use optional strict modes in review/CI where appropriate, but do not rewrite warnings as shipped guarantees. The gate may deliberately escalate a quality warning—such as an existence-only eval set—because autonomy demands a stronger bar than parseability.

PRE · CERTIFICATION

Safe to delegate

Only the gate can turn a reviewed contract into a dispatchable autonomy envelope.

`safe-to-delegate.sh` combines structural validation with an execution check of the eval mechanics. Without `--stamp`, it reports a verdict without writing Task-Spec or Converge metadata. The evals still execute, so their side effects must be controlled. With `--stamp`, it first removes any old signature; after all checks pass it writes `signed_off: true`, signer identity, timestamp, and—when a key and crypto provider exist—the HMAC seal.

- 1. Validate fields, zones, sizing, dependency, traceability, and shell shape.
- 2. Extract Success Criteria and the Exit Check into an isolated subshell.
- 3. Distinguish expected assertion failure from broken evaluation mechanics.
- 4. Write the envelope through the injection-safe frontmatter serializer.
- 5. Compute and verify the seal; report the resulting trust tier.

Before implementation, an eval that says “endpoint not found” may be behaving correctly. A syntax error, unbound variable, missing executable, or malformed integer comparison is not. PRE certifies that the task can fail for the right reason and that the authority surface is explicit.

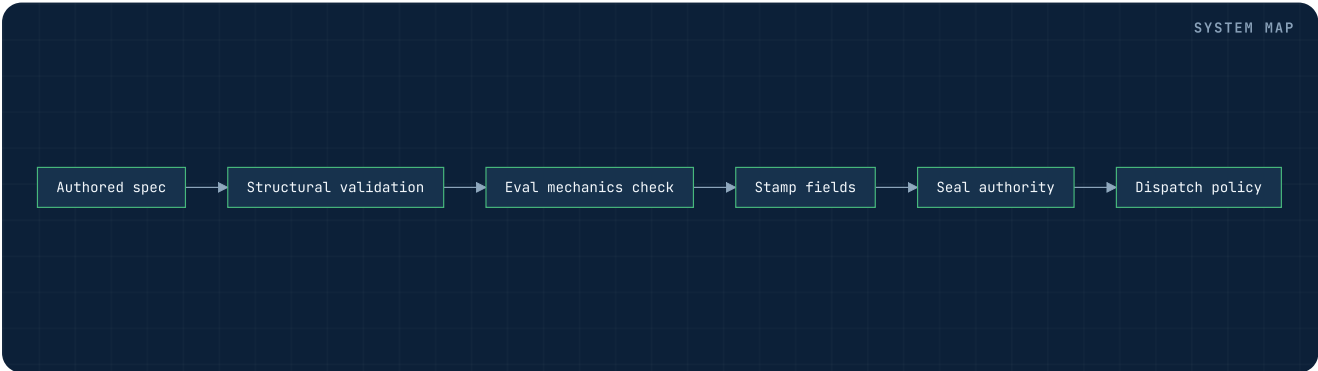
```
# Inspect only
cvg tasks gate cvg/tasks/T-20260729-add-health-endpoint.md

# Certify and seal
cvg tasks gate --stamp cvg/tasks/T-20260729-add-health-endpoint.md
```

Do not hand-stamp. A boolean written by an author or worker bypasses the evidence chain. Fix the contract until the gate accepts it, then let the gate write the envelope.

MODE	FILE MUTATION	USE
Without <code>--stamp</code>	None	Inspect verdict and repair contract
With <code>--stamp</code> , no key	Writes envelope	Tier 2; supervised dispatch only
With <code>--stamp</code> , key + crypto	Writes envelope and v2 sig	Tier 1 if immediate verification succeeds

Because removal happens before validation, a failed re-seal can leave a structurally signed task without its former cryptographic seal. Treat `--stamp` as a mutation even on failure, inspect the resulting tier, and do not dispatch until the fresh envelope verifies.



HMAC V2

Seal the authority, not just the prose

The current envelope detects post-sign changes to both the body and execution-critical frontmatter.

The legacy v1 envelope sealed the body and sign-off values but left execution-critical frontmatter outside the MAC. A signed task could have its write scope, dependencies, budget, or backend changed while the signature still verified. The implemented **hmac-sha256-v2** envelope closes that authorization gap.

The canonical payload contains:

- `id` and the SHA-256 digest of everything after frontmatter
- an authorization digest over, in fixed order: `touches_paths`, `creates_paths`, `depends_on`, `effort`, `execution_backend`, `agent`, `budget_iterations`, `budget_tokens`, and `requires`
- `signed_off`, `signed_off_by`, and `signed_off_at`

The field *names* are extracted in fixed order, but the captured field blocks are syntax-sensitive apart from trailing-whitespace trimming. Equivalent inline-versus-block YAML, indentation, list formatting, or body reflow can therefore change a digest. The signature field itself is excluded so it can be written after the payload is formed.

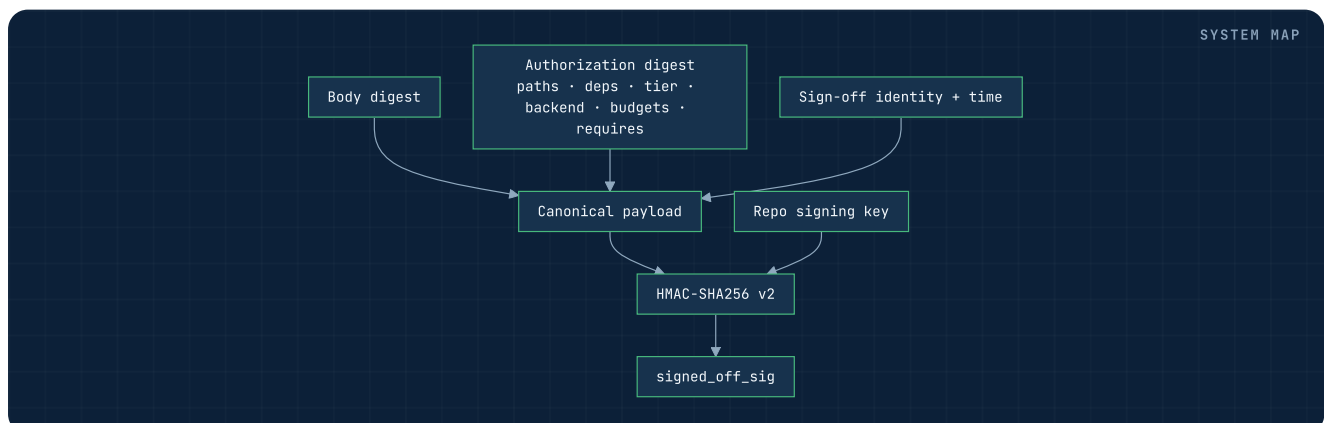
Lifecycle and receipt fields intentionally remain mutable: `status`, `accepted*`, `tracker_ref`, `signed_off_sig`, and projection metadata are not authority grants. Changing them for legitimate progress should not resemble tampering.

Compatibility: validator/PRE can recognize an authentic v1 seal as supervised Tier 2; keyed POST computes the current v2 envelope and rejects v1 until the spec is re-stamped. Newly stamped contracts use v2. Retained schemas and concept prose that show only v1 are known documentation drift, not the executable authority.

Why each field is sealed

Paths cap blast radius. Dependencies determine eligibility. Effort and backend decide routing. Agent and budgets constrain who may act and for how long. `requires` records the isolation posture the dispatcher must supply. Altering any of these after review changes authority even if not one word of the body moved.

The body digest hashes the extracted body text, so textual reformatting is significant as well as semantic edits. A worker cannot weaken an eval, remove Do-Not-Touch, or rewrite a promise without causing keyed verification to fail.



TRUST POLICY

Three tiers, no silent downgrade

Cryptographic availability changes supervision policy—not the meaning of a warning.

TIER	CONDITION	VERDICT	DISPATCH
1	Key present, v2 signature present, MAC verifies	Full repo-key trust	Unsupervised eligible
2	No key; key with no signature; key with a well-formed signature but no crypto provider; or authenticated legacy v1	Structural-only with loud warning	Supervised only
3	With a key present, malformed signature; or a recomputable MAC mismatch	Hard failure; treat as tampered	Refuse

Tier 2 exists so fresh clones and constrained environments remain usable, but it is not a green autonomy state. Without the key, a verifier cannot distinguish a legitimate stamp from a forged envelope. Supervision is the control that closes that downgrade path.

The key is symmetric. A valid Tier-1 MAC proves that a holder of the repo key sealed this contract; it does not prove which individual held the key. Anyone who can read it can forge a valid stamp. Per-author non-repudiation would require an asymmetric signature system and is outside the current contract.

The threat model is specific: accidental post-stamp edits and a co-author who knows the envelope shape but not the key. HMAC does not defend a machine already compromised by an attacker who can read both the repository and its secret.

Tier 3 recovery: inspect the changed body and authority fields, review the current contract, then run `cvg tasks gate --stamp <spec>` to create a new seal. Never “repair” the signature by hand.

Key operations

The setup utility creates 256-bit material, stores it outside version control—normally under the resolved Git directory with owner-only permissions—and prints a short key ID derived from its digest. CI may supply `TASKSPEC_SIGNING_KEY` through a secret manager. Never commit the key beside the task.

Verification looks for an existing key in the common Git directory first, then the current worktree Git directory; the setup utility still defaults new keys to the current git-dir. Trust decisions must use the key present in the actual dispatch environment, not assume that a stamp verified somewhere else will reach Tier 1 here.

EXECUTOR CONTRACT

Portable behavior, measurable conformance

A model name does not make an executor conformant; observable lifecycle behavior does.

A conformant executor reads the complete contract, claims work atomically, stays inside the declared authority, runs every eval, emits a permitted result, records attempts, and stops at the budget. The model, prompt strategy, and internal retrieval system may vary; the external behavior may not.

LEVEL	CUMULATIVE OBLIGATION	SAFE USE
L0	Read the spec and make its Exit Check pass	Supervised one-shot work
L1	L0 + honor lock and status lifecycle	Shared backlog
L2	L1 + honor retry budget and park with context	Unattended execution, subject to trust/profile policy

The bundled `ref-executor.sh` is a legacy/minimal fixture, not a current L1/L2 lifecycle implementation. It writes frontmatter status directly, without the portable state lock, POST acceptance, pass receipt, or locked `transition-status` path. Its small conformance scenarios are useful adapter examples, but they do not prove the complete lifecycle obligations listed here.

Each attempt emits one of `pass`, `fail`, `retry_with_reason`, or `parked_with_context`. A budget is authority, not advice: an L2 executor stops rather than purchasing progress forever.

```
bash skills/task-spec/scripts/conformance-check.sh --self-test
bash skills/task-spec/scripts/conformance-check.sh --level L2 --executor "my-runner --flags"
```

Dual certification: `safe-to-delegate` certifies a spec. `conformance-check` certifies an executor. Safe autonomy requires both claims to be true in the intended environment.

Normative observable L2 loop

1. Refuse any task that is not claimable under lifecycle and trust policy.
2. Acquire the lock and record `in-progress`.
3. Attempt work only inside the declared scope and runtime envelope.
4. Run every criterion, capture output/duration, emit one allowed result, and append a ledger record.
5. Retry only while budget and progress policy allow; otherwise park with the last useful diagnosis.

Conformance proves these behaviors against fixtures; it does not certify model quality, production credentials, or a particular task’s PRE envelope. Those remain separate controls.

POST · ACCEPTANCE

Prove the work, distrust the claim

Acceptance reruns evidence independently and checks that success was not purchased by widening scope or weakening proof.

`accept-task.sh` asks the inverse of the PRE gate: now that an executor says it is finished, is the result real? Its default gates are:

- 1. Evals pass.** The current Exit Check returns zero.
- 2. Scope is checked when available.** With Git data and the default policy, changed files must stay inside declared paths and outside Do-Not-Touch.
- 3. Contract is intact.** The PRE HMAC still verifies when keyed verification is available.
- 4. Isolation is reported.** Declared `requires` values are surfaced for operator confirmation; the shell does not claim to enforce them.

Current integrity boundary. POST attempts keyed HMAC verification when the key, signature, and crypto provider are available. If any is unavailable, the script reports structural-only integrity and may still accept. A deployment that requires Tier 1 must enforce that policy outside POST.

Scope caveat: `--no-blast-radius`, or a workspace where Git comparison is unavailable, can still reach acceptance with write scope unchecked. Use a Git-backed reviewed base when scope proof is part of the release claim.

With `--gold-sanity`, acceptance also runs the current evals against an unpatched baseline. The proof must fail there and pass on the current work. This detects non-discriminating or reward-hackable evals that would approve any implementation.

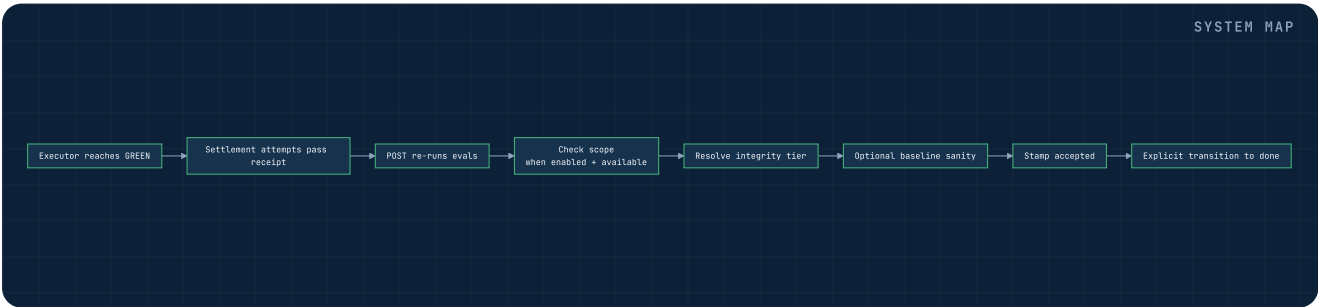
On success, `--stamp` writes `accepted: true`, `accepted_by`, and `accepted_at`, and appends an acceptance event to the ledger. These fields are the POST acceptance envelope, not part of the PRE authority MAC. They do not change `status`. A later explicit transition to `done` also requires `cvg/receipts/<id>.json` whose `task_id` matches and whose `result` is `pass`; the current guard does not authenticate who wrote it or validate its profile, hashes, or full schema.

```
cvgtasksaccept--gold-sanity--stamp--baseorigin/maincvg/tasks/T-20260729-add-health-endpoint.md
```

Done is the backstop: Loop or a harness should write the matching pass receipt, but release 0.1.0 does not revoke a settlement token solely when that write fails. POST changes no status; only `--stamp` writes its envelope. The locked transition independently refuses `done` until both files prove the minimum guard.

OPTION	EFFECT	EVIDENCE CAVEAT
<code>--stamp</code>	Writes accepted/by/at on success	Without it, verdict mode leaves no acceptance envelope
<code>--base REF</code>	Selects diff and fallback baseline reference	Use the reviewed integration base
<code>--gold-sanity</code>	Requires fail-before / pass-after discrimination	Warns if baseline cannot be reconstructed
<code>--no-blast-radius</code>	Skips scope comparison	Explicitly weakens the acceptance story

Gate D reports `requires`; it does not block on missing runtime proof. An operator or dispatcher must attach the sandbox evidence if that property matters to acceptance.



BACKLOG ARCHITECTURE

Five layers, one source of truth

Durable task files and append-only evidence make indexes disposable and crashes recoverable.

LAYER	PURPOSE	RECOVERY PROPERTY
1 • Git	Durable history once changes are committed	Clone or restore the repository
2 • <code>cvg/tasks/</code>	Active, done, and parked file layout	Human-readable backlog survives tooling
3 • Frontmatter	Authoritative per-task status and contract	Rebuild indexes from files
4 • <code>_state.yaml</code>	Fast derived index	Delete and regenerate when corrupt
5 • <code>_metrics.jsonl</code>	Append-only <code>created</code> , <code>accepted</code> , and <code>status_change</code> events	Lifecycle forensics; Loop attempts live under <code>cvg/loop/<id>/attempts/</code>

A transition validates the target enum, acquires a portable lock, and reads current frontmatter. A same-state request returns NOOP at that point. For an actual change, the tool rewrites or moves the task atomically, appends a metric event, then attempts a best-effort derived-index rebuild whose failure is suppressed. Release 0.1.0 does not enforce a source→target edge matrix. It adds the stamped acceptance and matching-pass-receipt guard only for an actual change whose target is `done`. Git preserves durable history only when the resulting files are committed; the state script does not commit automatically.

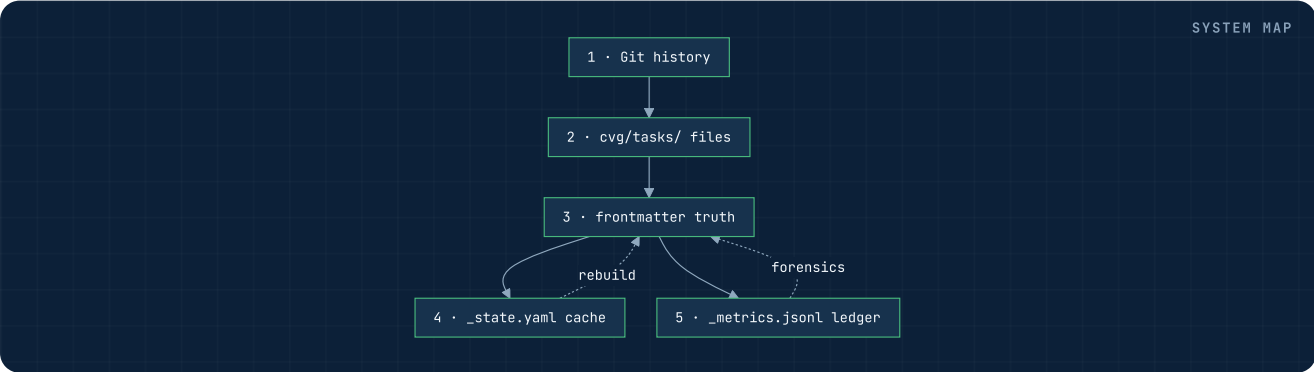
If `_state.yaml` disagrees with a task, rebuild it. If an executor crashes while `in-progress`, inspect lifecycle events together with Loop attempt logs, retained worktree, diff, and lock ownership; no recent-metric heuristic proves an executor is active. Then resume, block, or park explicitly. If task files are lost, restore from Git or backup, then rebuild the cache.

Architecture rule: never make the derived index more authoritative than the files it indexes. Caches can be fast precisely because they are replaceable.

Crash recovery sequence

- Find `in-progress` tasks and resolve their Loop workspace.
- Confirm no live executor owns the lock; remove only a proven stale lock.
- Read lifecycle events and attempt logs; inspect the retained worktree/diff.
- Choose resume, blocked, or parked; record a specific reason.
- Apply the transition atomically, append its event, and verify or rebuild `_state.yaml`.
- Before resuming a changed contract, rerun validation and PRE sealing.

If the task files themselves are lost, restore from Git or backup before touching caches. The metrics ledger aids forensics but cannot recreate an uncommitted contract body with certainty.



CONVERGE HANDOFF

Tasking → Register → Bind → Loop

The Task-Spec remains the scope-and-done authority while downstream artifacts add projection and runtime control.

PASS	CONSUMES	PRODUCES	AUTHORITY
5 • Tasking	Hardened plans	Signed Task-Spec DAG	Scope, constraints, and proof-of-done
6 • Register (opt-in)	Task-Specs	Tracker issues + backlink	Projection only; spec remains canonical
7 • Bind	One Task-Spec	Hash-bound runtime profile + compact task brief	Derived capability and enforcement envelope
8 • Loop	Spec + runtime contract	Settled work or explicit handoff	Bounded attempts, bound eval, and recorded Tier 2 verdict when configured

Register is optional and idempotent. The issue carries a stable marker; `tracker_ref` is written back as a convenience receipt when the environment permits. Editing an issue never silently changes the canonical Task-Spec.

Bind keeps authority ownership explicit. The Task-Spec remains canonical for scope, routing, budgets, and proof. Static binding validates source hashes, manifests, topology, and guards; it derives capability, policy, and enforcement artifacts plus a hash-bound pointer to canonical Task-Spec budgets. Credentials remain a host/harness concern, and separate `cvg doctor runtime-contract` probes current-host readiness.

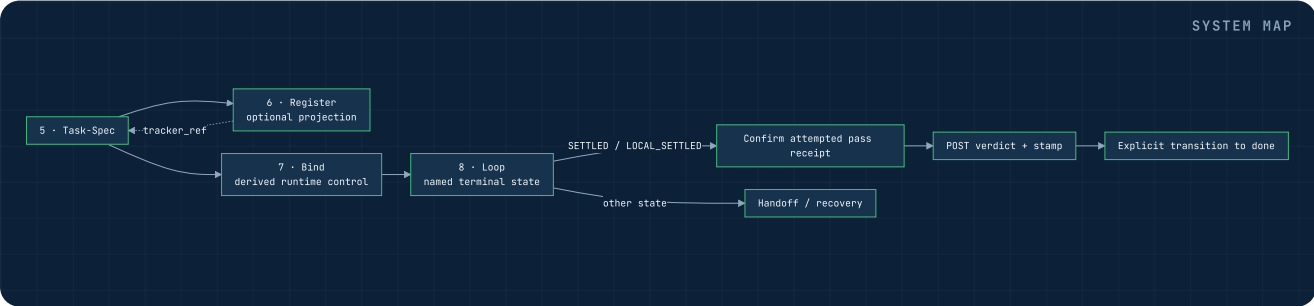
The worker brief is compact, not identifier-only: it includes IDs, Goal, exact allowed and forbidden paths, the actual Exit Check, network and external-write boundaries, assurance, and closure instructions.

Loop runs fresh attempts, evaluates progress and budgets, and applies any configured Tier 2 verifier before settlement, recording the judge and available independence label. Auto-selection compares judge family only with the Task-Spec’s recognized `execution_backend`, not the Loop `--agent`; an explicit or sole judge may be same-family. Text names judge and strength, JSON records `independence`, while Loop enforces the verdict token rather than strength. Settlement attempts the matching pass receipt last; a success token without that file is incomplete lifecycle evidence. POST remains separate: verdict without Task-Spec or Converge metadata writes, explicit acceptance stamp, then locked transition to `done`.

No duplicate truth: issue, runtime contract, worker brief, and loop receipt are projections for different consumers. The Task-Spec is still the canonical unit of scope and done.

CHANGE	CORRECT OWNER	REQUIRED FOLLOW-UP
Issue label/title	Tracker projection	No spec change unless canonical intent changed
Scope, dependency, budget, proof	Task-Spec	Review and fresh PRE stamp; resync projection
Allowed tools/commands/time	Runtime contract	Re-bind before another loop attempt
Attempt logs, verifier output, terminal state, and latest receipt subset	Loop evidence set	Preserve the logs; confirm a matching pass receipt before POST and the explicit done transition

This ownership table prevents a convenient delivery surface from silently becoming method truth.



AUTHORING WORKFLOW

From source evidence to accepted atom

A disciplined sequence prevents late discovery that the task was never atomic or provable.

- 1. Ground.** Read the source note, parent artifact, live code, and relevant tests. Record provenance.
- 2. Classify.** Decide leaf versus node, XS–XXL effort, and the lightest honest profile.
- 3. Bound.** Declare existing and new write paths, dependencies, Do-Not-Touch, backend, budgets, and isolation needs.
- 4. Write behavior.** Express stable B-N outcomes before choosing commands.
- 5. Design proof.** Write deterministic evals, mirror them in the Validation Card, map `verifies`, and compose the Exit Check.
- 6. Operationalize.** Add specific anti-patterns, open questions, and—on full—rollback and observability.
- 7. Validate and certify.** Repair every error, inspect warnings, then run the PRE gate with the intended signing environment.
- 8. Execute and receipt.** Use Loop or a conforming harness, preserve attempts, and require the matching pass receipt.
- 9. Accept and close.** Run POST without metadata writes, stamp only on a passing verdict, then explicitly transition to done.

```
# Scaffold → lint → PRE seal
cvg tasks new add-health-endpoint S
cvg tasks validate cvg/tasks/T-20260729-add-health-endpoint.md
cvg tasks gate --stamp cvg/tasks/T-20260729-add-health-endpoint.md

# Bind → Loop settlement → confirm passing receipt
cvg bind --task cvg/tasks/T-20260729-add-health-endpoint.md
cvg loop --issue T-20260729-add-health-endpoint --estimate
cvg loop --issue T-20260729-add-health-endpoint

# POST verdict → stamp → lifecycle done
cvg tasks accept cvg/tasks/T-20260729-add-health-endpoint.md
cvg tasks accept --gold-sanity --stamp --base origin/main cvg/tasks/T-20260729-add-health-endpoint.md
cvg transition T-20260729-add-health-endpoint done
```

`--estimate`, `--resume`, and `--gate-only` all still require `--issue`; none selects work. Estimate performs setup first, then reports configured iteration, per-attempt and elapsed-work limits, route/model/Tier-2 choices, and token brake; it is neither a currency forecast nor a strict end-to-end spend bound. Gate-only is the single-shot eval/path-policy settlement leg and can write receipts, commits, pushes, or PRs without global `--dry-run`.

Normal text-mode Loop dry-run may create setup/worktree lifecycle, then exits before worker/eval/path policy with `DRY_RUN=OK` only. Gate-only dry-run runs eval: GREEN previews path/settlement and emits `LOCAL_SETTLED|SETTLED`; RED exits 1 with a blocked report and no `TASK_LOOP`. Neither writes commit/push/PR/receipt effects. In 0.1.0, do not combine `--json --dry-run loop`: the wrapper drops dry-run and may execute a real run. Resume rechecks the current binding during eval preflight but shell-sources an unauthenticated `state.env` not tied to prior binding, base, workspace, checkpoint, or receipt; default fresh worktree resolution can restart at attempt 1.

Standalone `cvg eval`, `cvg verify`, and `cvg tasks accept` run level 1, tier 2, and POST respectively. None independently settles work or transitions lifecycle; full Loop coordinates attempts, proof, optional independent verification, and settlement.

Generate a scaffold only after the unit has a clear slug and effort. Scaffolding accelerates syntax; it does not make classification or proof decisions. Replace every placeholder, including “none” entries that are only template defaults.

Review the contract as an adversary: Can an executor make the eval green without implementing the behavior? Can it edit an undeclared file? Does a dependency or credential exist only in the author’s head? Would a crash leave enough state to recover? The strongest time to fix these gaps is before the autonomy seal.

Dry-run before mutating envelopes. Machine consumers should use documented exit codes and verdict tokens: validator success is structural, PRE `DELEGATE` is dispatch policy at the reported tier, and POST `ACCEPTED=1` is acceptance of the resulting work.

Final author review: confirm every path is exact, every dependency resolves, every B-N has proof, every eval can fail for the promised reason, every required tool/isolation need is declared, and every open question is either resolved or represented as blocked state. Then review the generated diff of the stamp itself.

TROUBLESHOOTING

Fail closed, repair the right layer

Error messages are evidence about contract shape, runtime proof, or trust—not generic friction.

SYMPTOM	LIKELY CAUSE	CORRECT REPAIR
ID/filename mismatch	Task renamed incompletely	Use one stable ID everywhere; update graph references deliberately
Missing path	Greenfield file under touches_paths	Move it to creates_paths
Orphan B-N	Promise lacks proof	Add a mapped eval or remove/refine behavior
PRE eval mechanics fail	Broken bash, missing command, unbound value	Make the eval portable and self-contained; do not suppress failure
L rejected	Backend not in long-horizon set	Choose/configure a capable backend or decompose
XL/XXL rejected	Too few children, or any node reached PRE	Create child leaves; release 0.1.0 does not separately validate node-owned write fields
Tier-3 HMAC	Body or authority changed after stamp	Review current contract and re-stamp; never edit the sig
POST blast breach	Work escaped declared scope	Revert escape or re-author/review/re-stamp expanded authority
Gold sanity fails	Evals pass on baseline	Strengthen proof so it discriminates the change
Stale in-progress	Executor crashed	Inspect ledger, release stale lock, resume/block/park explicitly, rebuild state

Avoid the inverted `grep -c` pattern that can yield malformed numeric output on zero matches. Prefer direct predicate commands whose exit status already means what the eval needs. Avoid making a failing proof green by adding `|| true`; that converts evidence into decoration.

Diagnose before retrying

- **Same failure, no diff:** likely stagnation or missing context; park instead of consuming the outer budget.
- **Command-not-found:** fix `required_tools` or runtime provisioning, not the assertion.
- **Dependency cycle:** redesign edges; manually marking a predecessor done corrupts lifecycle truth.
- **Baseline unavailable:** record gold-sanity as unchecked and obtain a resolvable ref before high-stakes acceptance.
- **Index drift:** rebuild from frontmatter; never “fix” task files to match cached state.

IMPLEMENTATION LEDGER

Current truth and known drift

The handbook is precise about what is implemented, compatible, advisory, or stale.

SURFACE	CURRENT IMPLEMENTED TRUTH	DRIFT TO IGNORE OR REPAIR
Release	Converge package 0.1.0	Old Task-Spec 3.6.0 PDF branding
Format	<code>format_version: 3</code>	Living files whose filenames still say v1
Profiles	<code>lite standard full</code>	Old <code>minimal</code> label
Zones	Six profile-scaled responsibilities	“Four zones” heading and inconsistent numbering in retained prose
L routing	Configurable long-horizon set; defaults include glm, claude, codex, kimi	Template/schema comments that say GLM only
HMAC	<code>hmac-sha256-v2</code> seals body + authority fields	Concept/schema text showing v1-only payload
Schema	Shell validator and library are operational authority	Published JSON Schema may lag newer auth/routing details
Isolation	<code>requires</code> is documented and reported	Shell cannot enforce image or egress policy

Compatibility behavior is not the same as preferred authoring. The validator continues to read legacy specs with warnings and can recognize older signatures, but a new autonomous task should use the current template, explicit profile, v2 seal when keyed, and current six-tier model.

Similarly, advisory guidance is not enforcement. Read-count ceilings, exact eval counts, and isolation policy require author or runtime judgment. The handbook labels them as such instead of upgrading prose into an imaginary gate.

Evidence boundary: when generated schema, prose, and code disagree, fix the documentation and schema—but do not claim the drifted artifact is what the executable gate enforces today.

Release verification checklist

Before describing Task-Spec as current, compare root `VERSION`, `cv9 version`, and changelog; inspect `TS_PROFILES`, size helpers, long-horizon defaults, authz field list, and signature formatter in `_lib.sh`; then read the live validator, PRE gate, and POST gate. Generated templates and schemas should be tested against those executables. A documentation-only match is not evidence that a gate enforces the same rule.

WORKED EXAMPLE

A health endpoint, end to end

A small standard-profile leaf shows how the layers compose without excess ceremony.

```
id: T-20260729-add-health-endpoint
format_version: 3
profile: standard
status: ready
effort: S
budget_iterations: 10
agent: any
depends_on: []
touches_paths:
  - src/api/server.py
  - tests/test_health.py
execution_backend: any
signed_off: false
accepted: false
```

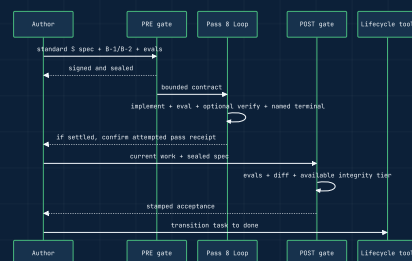
Intent: load-balancer probes already call `/health` and receive 404. The Goal is HTTP 200 with `{"status": "ok"}`. **Behavior:** B-1 covers the client response; B-2 covers the regression test. **Contract:** eval_1 starts or imports the app, eval_2 calls the endpoint and validates status/body, eval_3 runs the focused test. The Validation Card maps eval_1/eval_2 to B-1 and eval_3 to B-2; the Exit Check chains all three.

Bounds: only the server and focused test are writable. Deployment manifests are Do-Not-Touch because they already point to the correct route. Adding authentication or richer liveness semantics is an explicit anti-pattern for this atom.

Proof chain: structural validation confirms the S write budget and traceability. PRE runs the eval mechanics and stamps the contract. A conformant worker claims it, changes the two files, and runs the Exit Check. Pass 8 then applies its path and verification policy, settles, and attempts the matching pass receipt. This example proceeds only after that file exists. POST independently reruns the evals, checks the two paths and seal, and stamps acceptance; the locked transition can then move the task to `done`.

```
success_criteria:
- id: eval_1
  description: "Application starts cleanly"
  runnable: bash
  check_type: deterministic
  verifies: [B-1]
  terminal: true
  expected_duration_sec: 3
- id: eval_2
  description: "/health returns exact status and JSON"
  runnable: bash
  check_type: deterministic
  verifies: [B-1]
  terminal: true
  expected_duration_sec: 2
- id: eval_3
  description: "Focused regression suite passes"
  runnable: bash
  check_type: deterministic
  verifies: [B-2]
  terminal: true
  expected_duration_sec: 5
```

PRE should see eval_2 fail because the route is absent, not because `curl` or `jq` was omitted from required tools. POST should see the same function pass on current work and fail on the selected baseline. That is a complete discriminating story.



GLOSSARY & SOURCE MAP

Words that keep the boundaries clear

Use these terms precisely; they correspond to different evidence.

Valid — structurally acceptable to the current validator, possibly with compatibility warnings.

Signed off — PRE-certified by `safe-to-delegate`; dispatch policy still depends on trust tier.

Accepted — POST proof passed and the acceptance envelope was written.

Leaf — XS-L runnable Task-Spec with one done-condition.

Node — XL/XXL decomposition directive; never dispatched.

Profile — required documentation rigor: lite, standard, or full.

Effort — execution/decomposition size classification: XS through XXL.

Authority — paths, dependencies, routing, budgets, and isolation requirements a worker is permitted to exercise.

Receipt — durable evidence of a transition, projection, verification, or gate result.

PRE gate — certifies contract mechanics and writes the dispatch envelope; it does not certify implementation.

POST gate — reruns proof, conditionally checks the Git diff, and verifies keyed integrity when available; skipped scope or Tier-2 integrity is reported but may still accept.

Trust tier — policy result derived from key, signature, and verification state; not a quality score.

Ready frontier — `status: ready` tasks whose dependencies are done or legacy accepted. It does not validate leaf effort, PRE/trust, preconditions, or `blocked_reason`.

Runtime contract — Pass-7 capability envelope; it operationalizes but does not replace Task-Spec scope.

Operational source map

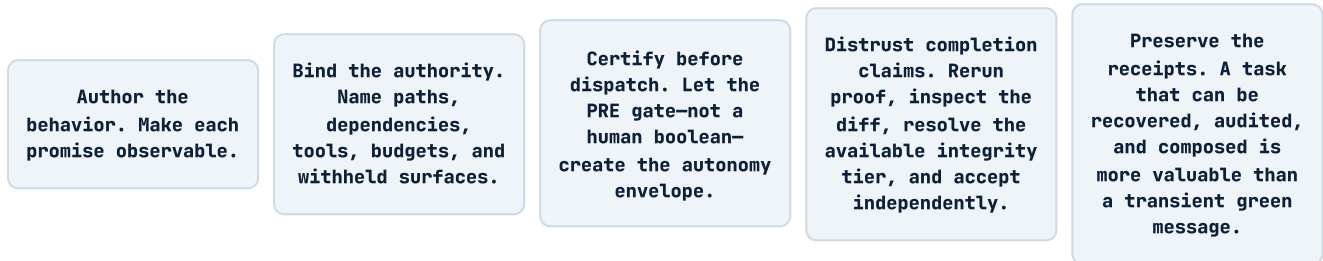
- `skills/task-spec/scripts/_lib.sh` — enums, profiles, crypto payload, state helpers
- `validate-task-spec.sh` — structural and traceability rules
- `safe-to-delegate.sh` — PRE verdict and sign-off envelope
- `accept-task.sh` — POST evidence and acceptance envelope
- `templates/task-spec.md.tpl` — current authoring scaffold, interpreted through live validators where comments drift
- `references/concepts/` and `runbooks/` — rationale, conformance, decomposition, backlog, recovery
- Root `VERSION` and `CHANGELOG.md` — package release truth

CLOSING

A trustworthy atom composes

The method works because each boundary is explicit, testable, and preserved as evidence.

Task-Spec does not ask an agent to be wise in the abstract. It gives a worker one bounded promise, an explicit authority surface, executable evidence, finite stopping conditions, and a lifecycle other systems can observe.



The result is small enough to execute, strong enough to verify, portable enough to outlive a model vendor, and honest enough to stop when the evidence is missing.

The unattended invariant: no leaf runs outside a bounded contract; no contract earns autonomy without PRE evidence; no execution earns acceptance without POST evidence; no large unit avoids decomposition by calling itself a task.

Explicit supervised migration modes may weaken the execution boundary; they are exceptions to unattended policy, not equivalent proof.

When any link is unavailable, the correct output is narrower authority: supervised dispatch, blocked input, parked execution, an unchecked acceptance note, or a refused verdict. The system becomes trustworthy by preserving those distinctions—not by forcing every artifact to look green.

